```
    const validStatus = validSeatStatus(seatNumber)
    if (validStatus === "invalid" || validStatus === "purchased") {
      return
    }
    const newSeatStatuses = updateSeatStatus(seatNumber)
    setSeatStatuses(newSeatStatuses)
»   props.subscription.perform("added_to_cart", {
»     concertId: props.concertId,
»     row: props.rowNumber + 1,
»     seatNumber: seatNumber + 1,
»     status: newSeatStatuses[seatNumber],
»     ticketsToBuyCount: props.ticketsToBuyCount,
»   })
  }
```

The subscription object is passed through as `props.subscription` and then we call the `perform` method on it. The first argument to `perform` is the name of the method to be invoked on the receiving channel, and the second argument is a JavaScript object that is received as the `data` argument to the method being invoked. In this case, we're passing all the same data that we would have been passing to the AJAX POST call.

And that is that. If you open two browsers side by side to the same concert page and click on a seat in one of them, the other browser will change to reflect that action almost immediately.

There is one other thing we can do, which is send the new ticket purchase back to our schedule page to adjust the number of tickets remaining to reflect the change. We have a couple of options here, depending on how much we want to upend the work we've already done in this channel. We could change the "sold out" messages we're sending to the JSON channel to update the amount of tickets available, and tweak our existing Stimulus code to handle that. Or we could use a Turbo Stream helper to automatically send a message when a Ticket is updated.

Both of these are viable options, but since we already have a Schedule channel to receive the data, we'll use that. However, later in Appendix 1, *Framework Swap*, when we rewrite the React page in Stimulus, we'll try it the other way.